Universidade Federal de Mato Grosso do Sul

Inteligência Artificial — turma P03, 2026-2

Classificação de dígitos no dataset SVHN

Relatório do trabalho final

Samuel Vieira da Cunha Nahas
Wilker Sebastian Afonso Pereira
Bernardo Willian de Sousa Pereira

Professor: Wesley Nunes Gonçalves

Novembro de 2026

Resumo

Este relatório descreve a classificação dos dígitos do SVHN com redes neurais em PyTorch, usando só os conjuntos `train` e `test`. Partimos de um classificador linear (30,29% de validação) e de uma MLP (86,25%) e chegamos a uma CNN de três blocos com data augmentation, cutout, weight decay e label smoothing (96,68%). Toda escolha foi feita pela validação, numa divisão fixa do `train` com a semente 42. A solução final é um ensemble de 20 dessas CNNs, treinadas por 60 épocas com sementes fixas e avaliadas com deslocamentos de 1 e 2 pixels (TTA). Ela tem 97,52% de validação e 97,89% no teste, e é o modelo que gerou as predições do conjunto adicional. As mudanças de maior impacto foram trocar a MLP pela CNN, normalizar a entrada, usar data augmentation e combinar vários modelos.

1 Solução desenvolvida

Protocolo. O `split="train"` (73.257 imagens) foi dividido em 90% para treino (65.932) e 10% para validação (7.325), sempre com a semente 42. A normalização usa a média e o desvio de cada canal calculados só no treino. Cada modelo é avaliado na época de maior acurácia de validação. O `split="test"` (26.032 imagens) só informa a acurácia e não participa de nenhuma escolha, e o conjunto adicional do professor só foi usado depois que a solução final estava definida. Todos os modelos usam batch 64, taxa de aprendizado com decaimento cosseno ao longo das épocas e entropia cruzada, salvo quando o próprio experimento muda isso.

Semente. A semente 42 fixa a divisão treino/validação, a inicialização dos pesos, a ordem dos lotes e o augment, de modo que rodar de novo dá o mesmo resultado e todos os experimentos são comparados nas mesmas 7.325 imagens de validação. No ensemble, o membro i usa a semente $42 + i$ (42 a 61), numa ordem fixada antes de ver os resultados. A divisão continua sendo sempre a da semente 42.

Classificador linear. Uma camada linear de $3 \cdot 32 \cdot 32 = 3072$ entradas para 10 saídas (30.730 parâmetros), com softmax e entropia cruzada. Foi treinado com Adam, taxa de aprendizado 10^{-3}, por 15 épocas. Também testamos a loss SVM multiclasse da aula.

MLP. Duas camadas ocultas de 512 e 256 neurônios com ReLU (1.707.274 parâmetros), com as mesmas condições do linear: Adam, taxa de aprendizado 10^{-3}, batch 64 e 15 épocas. A partir dela variamos a taxa de aprendizado, o batch, o otimizador (SGD, com e sem momentum), as ativações, a normalização da entrada, a profundidade e a regularização (L2, Dropout e BatchNorm).

CNN. Três blocos de convoluções 3×3 com BatchNorm e ReLU, seguidos de max pooling (32, 64 e 128 filtros; a imagem vai de 32×32 a 4×4), e uma camada densa de 256 com Dropout 0,3. Sobre ela testamos data augmentation (rotação de $\pm 8°$, escala de $\pm 10\%$ e translação de $\pm 8\%$, sem espelhamento, porque um dígito espelhado é outro símbolo), filtros mais largos, weight decay, mais épocas, label smoothing, uma terceira convolução por bloco e cutout (um quadrado de 8×8 zerado em posição aleatória).

Hiperparâmetros mais importantes. Pelos experimentos, os que mais pesaram foram a taxa de aprendizado (com Adam, 10^{-2} impede o treino da MLP), a normalização da entrada, o data augmentation, o número de épocas e, na solução final, o número de modelos do ensemble. Batch, ativação ReLU contra Leaky ReLU e largura da CNN mudaram pouco.

Código. O notebook `svhn.ipynb` tem todos os experimentos das linhas 1 a 30 da Tabela 1, o ensemble de 5, a solução final e as predições. Os treinos em lote das linhas 31 em diante usaram o script `dgx/svhn_dgx.py`, com o mesmo protocolo, a mesma divisão e a mesma normalização do notebook. O script confere a divisão antes de treinar.

Organização e metodologia. O trabalho foi feito no GitHub, em `https://github.com/SamuelNahas/svhn-classificacao`, onde estão o código, os pesos, os resultados de cada experimento e este relatório. Cada entregável virou uma issue, com as decisões do grupo registradas numa issue principal, e o andamento foi acompanhado num board, em `https://github.com/users/SamuelNahas/projects/2` (Backlog, Fazendo, Verificação e Pronto). Cada issue teve a sua branch e um pull request revisado antes de entrar. A cada pull request, a integração contínua executa o notebook inteiro num modo de teste rápido e confere as saídas. Nos experimentos, cada rodada mudou uma coisa por vez, e o plano e a regra de decisão de cada rodada foram registrados no repositório antes de rodar.

2 Experimentos

A Tabela 1 traz os experimentos principais. Cada um muda uma coisa em relação a outro, e a última coluna compara a acurácia de validação com essa base, indicada pelo número da linha. As métricas são as da melhor época de validação. A escolha sempre foi feita pela validação.

Tabela 1: Experimentos. Loss: entropia cruzada. Acurácias em %. Δ: diferença de acurácia de validação, em pontos, em relação à linha indicada entre parênteses.

#	Experimento	Modificação	Loss		Acurácia			Δ val (base)
			treino	val.	treino	val.	teste	
Notebook: linear, MLP e CNN (15 épocas, salvo indicação)								
1	`linear`	classificador linear 3072→10, softmax	2,081	2,134	30,97	30,29	25,30	
2	`linear_svm`	loss SVM multiclasse	2,101	2,153	28,33	27,58	23,64	−2,72 (1) piorou
3	`mlp_baseline`	MLP 512-256, ReLU, Adam lr 10^{-3}	0,179	0,560	95,08	86,25	82,88	
4	`mlp_lr_1e-4`	lr 10^{-4}	0,325	0,490	91,21	86,62	84,27	+0,37 (3) melhorou
5	`mlp_lr_1e-2`	lr 10^{-2}	2,238	2,225	18,83	19,70	19,59	−66,55 (3) piorou
6	`mlp_batch_32`	batch 32	0,220	0,557	93,52	85,76	82,73	−0,49 (3) piorou
7	`mlp_batch_256`	batch 256	0,195	0,520	94,95	86,59	83,87	+0,34 (3) melhorou
8	`mlp_sgd`	SGD (lr 10^{-2})	0,456	0,554	87,27	84,66	82,33	+64,96 (5) melhorou
9	`mlp_sgd_momentum`	+ momentum 0,9	0,166	0,510	95,66	86,88	84,24	+2,23 (8) melhorou
10	`mlp_100`	1 camada oculta de 100	0,366	0,518	89,83	85,84	83,77	−0,41 (3) piorou
11	`mlp_leaky_relu`	Leaky ReLU	0,165	0,530	95,78	86,80	84,12	+0,55 (3) melhorou
12	`mlp_tanh`	tanh	0,545	0,801	83,61	75,41	72,23	−10,84 (3) piorou
13	`mlp_sigmoid`	sigmóide	0,387	0,597	88,56	81,69	79,09	−4,56 (3) piorou
14	`mlp_sem_norm`	sem normalização (só /255)	1,486	1,507	50,42	49,76	46,74	−36,49 (3) piorou
15	`mlp_so_media`	só subtrai a média	0,169	0,528	95,55	86,10	83,85	−0,15 (3) piorou
16	`mlp_cinza`	tons de cinza	0,175	0,529	95,19	86,66	83,65	+0,41 (3) melhorou
17	`mlp_l2`	L2, $\lambda = 10^{-4}$	0,258	0,499	92,86	86,31	83,64	+0,05 (3) melhorou
18	`mlp_dropout`	Dropout 0,3	0,418	0,530	87,38	84,60	81,61	−1,65 (3) piorou
19	`mlp_dropout_0.5`	Dropout 0,5	0,597	0,657	81,86	80,26	76,79	−4,34 (18) piorou
20	`mlp_bn_dropout`	+ BatchNorm	0,352	0,449	89,77	86,46	84,31	+1,86 (18) melhorou
21	`mlp_deep`	camadas 1024-512-256	0,324	0,439	90,15	86,78	84,17	+0,33 (20) melhorou
22	`cnn`	CNN 3 blocos 32/64/128	0,035	0,204	99,24	95,33	95,21	
23	`cnn_augment`	+ data augmentation	0,112	0,167	97,27	95,59	96,05	+0,26 (22) melhorou
24	`cnn_wide`	filtros 64/128/256	0,117	0,175	97,36	95,48	96,12	−0,11 (23) piorou
25	`cnn_wd`	weight decay 0,05	0,105	0,153	97,46	95,93	96,22	+0,34 (23) melhorou
26	`cnn_30ep`	30 épocas	0,060	0,161	98,54	96,01	96,37	+0,42 (23) melhorou
27	`cnn_ls`	label smoothing 0,1	0,192	0,230	97,31	95,81	96,51	+0,22 (23) melhorou
28	`cnn_deep`	3 convoluções por bloco	0,089	0,153	97,88	96,03	96,37	+0,44 (23) melhorou
29	`cnn_cutout`	cutout 8×8	0,128	0,163	96,87	95,82	96,00	+0,23 (23) melhorou
30	`cnn_best`	augment + as 5 mudanças que ajudaram	0,135	0,198	98,78	96,68	97,05	+1,09 (23) melhorou
Triagem em lote: uma mudança por vez sobre a `cnn_best` (semente 42, 30 épocas, salvo indicação)[a,b]								
31	`t01_ref_bf16`	referência: `cnn_best` (bf16)	0,123	0,190	98,98	96,66	97,11	
32	`t02_ref_fp32`	fp32	0,126	0,192	98,91	96,56	97,14	−0,10 (31) piorou
33	`t12_ep60`	60 épocas	0,114	0,195	99,10	96,66	97,16	+0,00 (31) igual
34	`t03_wrn16_4`	Wide ResNet 16-4	0,127	0,198	99,21	97,08	97,46	+0,42 (31) melhorou
35	`t04_wrn16_4_sgd`	WRN-16-4 com SGD	0,150	0,188	98,35	97,01	97,51	+0,35 (31) melhorou
36	`t05_wrn22_1`	ResNet pequena (WRN-22-1)	0,142	0,197	98,57	96,63	97,27	−0,03 (31) piorou
37	`t06_ema`	média móvel dos pesos (EMA)	0,159	0,198	98,02	96,68	97,12	+0,03 (31) melhorou
38	`t07_color`	brilho/contraste ±20%	0,130	0,193	98,88	96,83	97,09	+0,18 (31) melhorou
39	`t08_geo05`	augment geométrico ×0,5	0,104	0,196	99,40	96,60	96,89	−0,05 (31) piorou
40	`t09_geo15`	augment geométrico ×1,5	0,170	0,211	97,79	96,42	96,88	−0,23 (31) piorou
41	`t10_cut12`	cutout 12×12	0,144	0,189	98,44	96,79	97,24	+0,14 (31) melhorou
42	`t11_cutp05`	cutout com $p = 0{,}5$	0,119	0,196	99,13	96,59	96,98	−0,07 (31) piorou
43	`t15_wrn16_4_color`	WRN-16-4 + brilho/contraste	0,126	0,195	99,18	96,83	97,43	+0,18 (31) melhorou

#	Experimento	Modificação	Loss		Acurácia			Δ val (base)
			treino	val.	treino	val.	teste	
44	`t16_wrn16_4_ep60`	WRN-16-4, 60 épocas	0,112	0,202	99,56	96,85	97,63	+0,19 (31) melhorou
45	`t17_wrn16_8`	WRN-16-8	0,116	0,199	99,52	96,91	97,43	+0,26 (31) melhorou
46	`t18_wrn28_4`	WRN-28-4	0,123	0,188	99,13	96,86	97,51	+0,20 (31) melhorou
Confirmação com várias sementes[a,b]								
47		WRN-16-4, 30 épocas (média de 3 sementes)	0,126	0,196	99,21	96,98	97,50	
48		WRN-16-4, 60 épocas (média de 3)	0,114	0,204	99,55	96,86	97,60	−0,13 (47) piorou
49		CNN, 60 épocas (média de 3: s42–44)	0,106	0,195	99,42	96,79	97,21	
50		+ RandAugment no lugar do augment (média de 3)	0,101	0,186	99,38	96,91	97,03	+0,12 (49) melhorou
51		+ AutoAugment SVHN (média de 3)	0,117	0,191	99,17	96,86	97,27	+0,07 (49) melhorou
52		+ TrivialAugment (média de 3)	0,109	0,196	99,33	96,68	97,05	−0,10 (49) piorou
53		divisão 43: CNN, 30 épocas (média de 10)	0,121	0,196	99,10	96,68	97,12	
54		divisão 43: CNN, 60 épocas (média de 10)	0,101	0,196	99,60	96,78	97,10	+0,10 (53) melhorou
TTA e ensembles (média simples do softmax; membros em ordem fixa de sementes)[b,c]								
55		1 `cnn_best` + TTA ±2 px	—	—	—	96,76	97,11	+0,08 (30) melhorou
56		ensemble de 5 `cnn_best` (30 épocas)	—	—	—	97,06	97,51	+0,38 (30) melhorou
57		ensemble de 5 `cnn_best` + TTA ±2 px (PR #21)	—	—	—	97,31	97,57	+0,25 (56) melhorou
58		ensemble de 5 CNNs de 60 épocas	—	—	—	97,39	97,71	+0,33 (56) melhorou
59		ensemble de 10 CNNs de 60 épocas	—	—	—	97,41	97,73	+0,01 (58) melhorou
60		ensemble de 20 CNNs de 60 épocas	—	—	—	97,47	97,75	+0,07 (59) melhorou
61		ensemble de 30 CNNs de 60 épocas	—	—	—	97,42	97,76	−0,05 (60) piorou
62		ensemble de 10 CNNs de 60 épocas + TTA shift12	—	—	—	97,38	—	−0,03 (59) piorou
63		ensemble de 10 WRN-16-4 + TTA shift12	—	—	—	97,28	—	−0,10 (62) piorou
64		ensemble de 20 CNNs de 60 épocas + TTA shift12	—	—	—	97,52	—	+0,04 (60) melhorou
65		ensemble misto WRN + CNN (20) + TTA shift12	—	—	—	97,46	—	−0,05 (64) piorou
66		**solução final: os 20 da linha 64 retreinados no treino inteiro**	—	—	—	**97,52**	**97,89**	escolhida (64)

[a] Nos treinos em lote, a loss e a acurácia de treino são medidas numa amostra fixa de 7.325 imagens do treino, para economizar tempo; a validação é sempre completa. [b] A acurácia de teste das linhas 31 em diante foi calculada durante os treinos, mas só foi lida depois de a solução final estar fechada. Ela não influenciou nenhuma escolha. [c] Ensembles não têm métricas de treino (—). O teste das linhas com TTA shift12 só foi calculado para a solução final; o das outras linhas de ensemble de 60 épocas é sem TTA. TTA ±2 px: a imagem e 4 cópias deslocadas 2 pixels; shift12: a imagem e 8 cópias deslocadas 1 e 2 pixels.

O que os experimentos mostram.

- **Família de modelo.** Linear, MLP e CNN são os maiores saltos: 30,29%, 86,25% e 95,33% na validação. O linear sobre pixels crus não separa os dígitos, e a CNN ganha por aproveitar a estrutura espacial da imagem.
- **MLP.** A normalização é essencial (sem ela, a validação cai para menos de 50%), a taxa de aprendizado 10^{-2} com Adam impede o treino, e tanh e sigmóide aprendem pior que ReLU. O SGD com momentum foi a melhor MLP (86,88%). Dropout sozinho reduz a diferença entre treino e validação, mas perde acurácia; com BatchNorm, a acurácia volta e a diferença cai para cerca de 3 pontos.
- **CNN.** Data augmentation, weight decay, 30 épocas, label smoothing, a terceira convolução por bloco e o cutout melhoraram a validação isoladamente, e a `cnn_best` junta as seis. Filtros mais largos (4 vezes os parâmetros) pioraram e ficaram de fora.

- **Triagem em lote.** A Wide ResNet 16-4 foi a única mudança grande num modelo sozinho; redes residuais maiores não a superaram. EMA e mudar a intensidade do augment não ajudaram.
- **Épocas.** Com várias sementes, 60 épocas ganham de 30 na CNN, também numa segunda divisão que não guiou nenhuma escolha (96,68% → 96,78%). Na WRN-16-4, 60 épocas pioraram (96,98% → 96,86%). O augment automático (RandAugment, AutoAugment e TrivialAugment) não ganhou o bastante (a regra registrada exigia +0,15 ponto na média e a maioria dos pares).
- **Ensemble e TTA.** Combinar modelos foi o que mais rendeu depois da CNN. A curva da CNN de 60 épocas sobe até 20 membros e para (30 membros não ganham). A WRN, melhor sozinha, perde no ensemble, e o ensemble misto de WRN e CNN também não superou o de CNNs.

3 Curvas de treinamento

As Figuras 1 a 5 mostram a loss e a acurácia de treino e validação por época dos principais modelos.

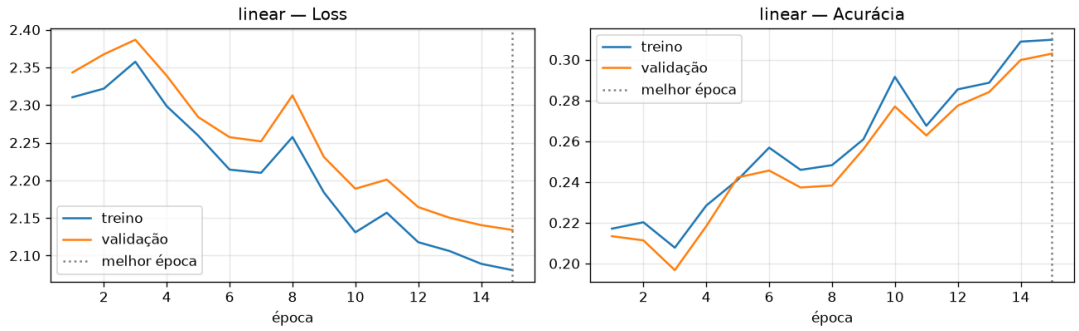


Figura 1: Classificador linear. Treino e validação ficam juntos e baixos (cerca de 30%): é **underfitting**, porque o modelo não tem capacidade para separar os dígitos.

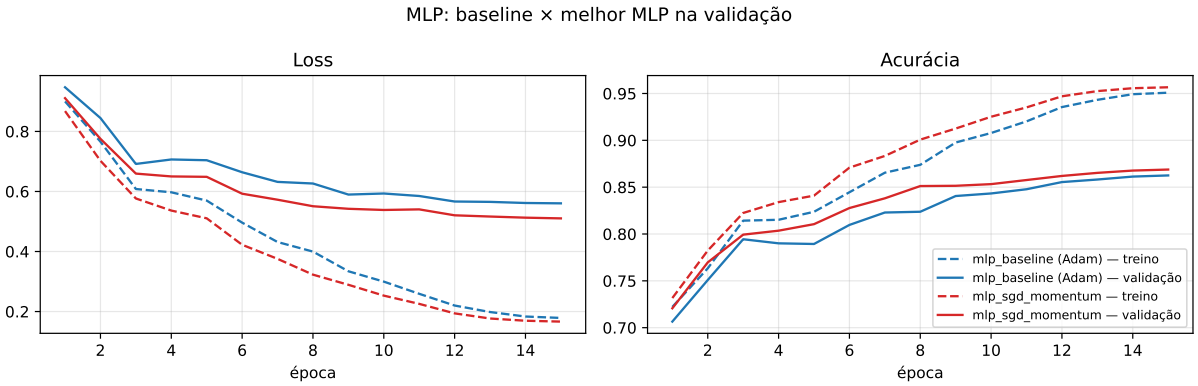


Figura 2: MLP baseline (Adam) e a melhor MLP na validação (SGD com momentum). A loss de treino continua caindo enquanto a de validação estabiliza, e a acurácia de treino passa da de validação por cerca de 9 pontos: **overfitting**.

Overfitting, underfitting e instabilidade. O linear tem underfitting (30,97% de treino e 30,29% de validação). A MLP baseline tem overfitting forte (95,08% de treino contra 86,25% de validação), que Dropout e BatchNorm reduzem (para cerca de 3 pontos), mas sem que a validação passe de 87%. A CNN sem augment também tem overfitting (99,24% contra 95,33%), e a regularização da `cnn_best` reduz essa diferença. A única instabilidade séria foi a MLP com Adam e taxa de aprendizado 10^{-2}: a loss fica parada em 2,24 desde a primeira época e a acurácia em cerca de 20% (linha 5). Com SGD e a mesma taxa, o treino converge.

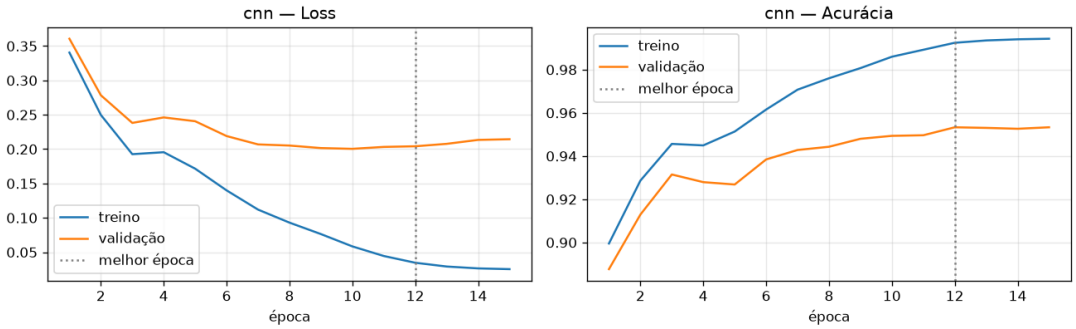


Figura 3: CNN sem augment. O treino chega perto de 100%, e a loss de validação atinge o mínimo na época 10 e volta a subir: **overfitting**, que o data augmentation corrige nas versões seguintes.

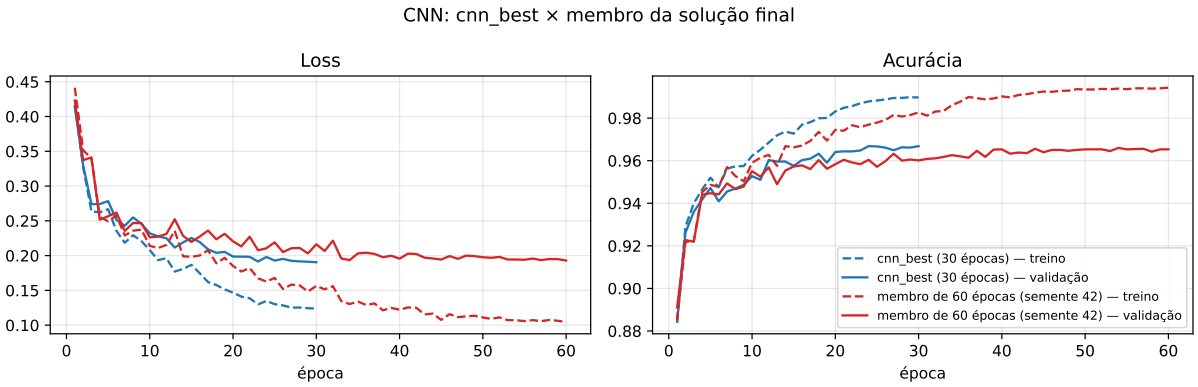


Figura 4: A `cnn_best` (30 épocas) e um membro da solução final (a mesma CNN, 60 épocas, semente 42). Com augment, cutout, weight decay e label smoothing, a diferença entre treino e validação é menor que na Figura 3 e a loss de validação não volta a subir. No membro de 60 épocas, as métricas de treino são medidas numa amostra fixa de 7.325 imagens do treino.

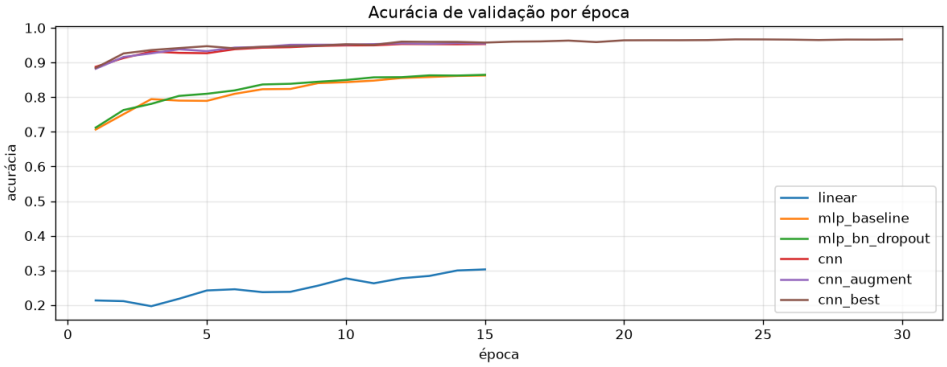


Figura 5: Acurácia de validação por época dos principais modelos.

4 Melhor solução

Arquitetura. Um ensemble de 20 CNNs iguais à `cnn_best`: três blocos de três convoluções 3×3 (32, 64 e 128 filtros) com BatchNorm e ReLU, max pooling depois de cada bloco, e uma camada densa de 256 com Dropout 0,3 antes da saída de 10 classes. Cada CNN tem 1.009.226 parâmetros, e o ensemble, 20.184.520. A predição é a média do softmax dos 20 modelos na imagem original e em 8 cópias deslocadas 1 e 2 pixels na horizontal e na vertical (TTA shift12), sem espelhamento.

Hiperparâmetros. AdamW com taxa de aprendizado 10^{-3}, weight decay 0,05, decaimento cosseno em 60 épocas, batch 64, label smoothing 0,1, data augmentation geométrico e cutout 8×8. As sementes são 42 a 61, uma por membro, e a divisão é sempre a da semente 42.

Treino final. Cada membro foi retreinado com todo o `split="train"` (treino e validação), como o enunciado permite, e parou na época em que o membro de validação da mesma semente teve a melhor validação (entre as épocas 40 e 60).

Acurácia. 97,52% de validação (linha 66 da Tabela 1) e **97,89% no teste**. A solução anterior, o ensemble de 5 `cnn_best` com TTA, tinha 97,31% e 97,63%, e uma única `cnn_best` retreinada, 96,68% e 97,13%.

Motivo da escolha. Foi a maior acurácia de validação entre todas as soluções testadas, e a regra de escolha foi registrada antes de ver os resultados. O número de membros é o menor em que a curva de validação já estabilizou (20; com 25 e 30, não ganha), e o TTA é o melhor na validação com 20 membros. Mais épocas também ganharam numa segunda divisão, que não guiou nenhuma escolha. O teste foi lido uma única vez, já com a solução fechada.

Predições e reprodução. É este o modelo que gerou o arquivo `SAMUEL_WILKER_BERNARDO_RESULTS.txt`: o notebook carrega os pesos dos 20 membros (`weights/pt4/`) e classifica automaticamente as 203.856 imagens do conjunto adicional, que nunca foram usadas no treino nem em nenhuma escolha. As acurácias e as predições se reproduzem exatamente a partir desses pesos, pelo notebook. Retreinar os membros com as mesmas sementes pelo script reproduz o resultado dentro de cerca de 0,1 ponto, porque o treino na GPU não é determinístico bit a bit.

5 Conclusão

O que funcionou melhor. A CNN regularizada, combinada num ensemble. Sozinha, a `cnn_best` chega a 96,68% de validação; o ensemble de 20 com TTA, a 97,52%, e 97,89% no teste.

Maior impacto. Em ordem: trocar a MLP pela CNN (de 86,88% para 95,33%); normalizar a entrada (sem ela, a MLP perde mais de 30 pontos); usar uma taxa de aprendizado adequada; data augmentation e as demais regularizações da `cnn_best`; e o ensemble com TTA, que reduziu o erro de teste de uma CNN sozinha em cerca de um quarto. Mudanças de capacidade (filtros mais largos e redes residuais maiores) renderam menos que regularizar e combinar modelos.

Dificuldades. Reduzir o overfitting da MLP sem perder acurácia de validação; a instabilidade do Adam com taxa alta; o custo de treino, já que a solução final tem 20 modelos de 60 épocas, o que nos levou a treinar vários modelos em paralelo na GPU; separar ganhos reais do ruído entre sementes (cerca de 0,1 ponto), o que exigiu repetir as melhores mudanças com mais sementes e registrar as regras de decisão antes de ver os resultados; e diferenças pequenas de predição entre a GPU e o CPU, que tornaram necessário gerar as predições finais sempre pelo mesmo código.